

Sistema de Busca para Jogos Usando Trie

José Pedro Dresch

Phelipe Gabriel Lima da Silva

1 Representação da Trie

A Trie foi implementada com um alfabeto de tamanho 36, composto por 26 letras minúsculas (a–z) e 10 algarismos (0–9). Antes de qualquer inserção ou busca, os títulos e prefixos são convertidos para uma representação interna denominada chave de busca, obtida pelo método `toSearchKey`.

O mapeamento de cada caractere para um índice no vetor de filhos segue a tabela ASCII. Ou seja, letras minúsculas seguem o índice $i = c - 97$, onde c é o código ASCII do caractere. Assim, 'a' mapeia para 0, 'b' para 1, e assim até 'z' para 25. Para algarismos, o índice é $i = c - 22$, onde c é o código ASCII do dígito. Logo, '0' mapeia para 26, '1' para 27, e assim até '9' para 35.

O método `toSearchKey` percorre o texto caractere a caractere e produz uma nova string segundo as regras: Letras maiúsculas são convertidas para minúsculas somando 32 ao código ASCII; Letras minúsculas e algarismos são copiados diretamente; Espaços em branco e quaisquer outros símbolos são ignorados.

Isso garante que a busca seja case-insensitive e que espaços em branco sejam desconsiderados. Por exemplo, os títulos "Half Life", "halflife" e "HALF LIFE" produzem todos a mesma chave "halflife", e os prefixos "ha", "HA" e "Ha" produzem todos a mesma chave "ha".

2 Funcionamento dos Métodos

O método `insert` recebe um ponteiro para um objeto `Game` e insere o jogo na Trie usando a chave de busca derivada do seu título. Para cada caractere da chave, calcula o índice correspondente no vetor de filhos do nó atual; se o filho não existe, um novo `TrieNode` é criado. Ao final, o nó correspondente ao último caractere é marcado como fim de título e recebe o ponteiro para o jogo. Caso o título já exista na Trie, a inserção é recusada e o método retorna `false`.

O método `contains` converte o título recebido para chave de busca e percorre a Trie seguindo os caracteres. Se em qualquer ponto o filho correspondente ao caractere atual não existir, retorna `false`. Ao final da travessia, retorna o valor de `isEndOfTitle` do nó alcançado, indicando se aquele caminho corresponde a um título completo.

O método `autocomplete` recebe um prefixo e um inteiro k . Se $k \leq 0$, retorna imediatamente um vetor vazio. Caso contrário, converte o prefixo para chave de busca e caminha pela Trie até o nó correspondente ao último caractere do prefixo. Se em algum

momento o caminho não existir, retorna um vetor vazio. A partir do nó encontrado, o método auxiliar `find_games` coleta recursivamente todos os jogos da subárvore. O vetor de jogos é então ordenado por `sortResults` e, se houver mais de k jogos, é truncado para os k primeiros.

O método `sortResults` ordena o vetor de jogos usando o algoritmo Insertion Sort. O critério de ordenação tem dois fatores: primeiro por popularidade decrescente e, em caso de empate, por ordem alfabética crescente da chave de busca do título.

3 Análise de Custo

Parâmetros utilizados: L sendo comprimento do título após conversão para chave de busca; p sendo comprimento do prefixo após conversão para chave de busca; s sendo número de nós visitados na subárvore do prefixo; r sendo o número de jogos encontrados para o prefixo buscado.

Método insert: A inserção percorre exatamente L nós da Trie, criando filhos quando necessário. O custo é $O(L)$.

Método contains: A busca percorre no máximo L nós. O custo é $O(L)$.

Método autocomplete: A travessia até o nó do prefixo custa $O(p)$. A coleta de todos os jogos pela função `find_games` visita s nós, custando $O(s)$. A ordenação por Insertion Sort sobre r jogos tem custo $O(r^2)$ no pior caso. Assim, o custo total do `autocomplete` é $O(p + s + r^2)$. Na prática, r é limitado a k após o truncamento, mas a ordenação ocorre antes, sobre todos os r jogos encontrados. Como r pode ser muito menor que s (muitos nós podem não terminar em jogos), e k é geralmente pequeno, o custo na prática é bem inferior ao pior caso teórico.

4 Comparação e Uso de Memória

4.1 Trie versus busca linear

Uma solução ingênua que percorre o array original de jogos para cada chamada de `autocomplete` teria custo $O(N \cdot L)$ por consulta, onde N é o número total de jogos e L é o comprimento médio dos títulos. Com a Trie, o custo de chegar ao nó do prefixo é $O(p)$ e a coleta de candidatos é $O(s)$, sendo $s \ll N$ para prefixos suficientemente específicos.

4.2 Custo de memória

Cada `TrieNode` armazena um vetor de 36 ponteiros (para filhos), um booleano e um ponteiro para `Game`. O consumo de memória é proporcional ao número total de nós da Trie, que depende do vocabulário dos títulos. No pior caso, com títulos completamente

distintos entre si, o número de nós é proporcional a $N \cdot L$. No entanto, o compartilhamento de prefixos reduz significativamente esse número.

4.3 Compartilhamento de prefixos

Títulos com prefixos comuns, como “Counter Strike Global Offensive” e “Counter Strike Source”, compartilham os nós correspondentes ao prefixo "**counterstr**" e seguintes. Isso reduz tanto o número de nós quanto o tempo de busca em comparação com estruturas que armazenam cada título de forma independente, como uma lista ou um array de strings.

4.4 Impacto do tamanho do alfabeto

Cada nó aloca espaço para 36 ponteiros, independentemente de quantos filhos realmente existem. Para um alfabeto de tamanho A , cada nó consome $O(A)$ de memória. Alfabetos maiores aumentam o consumo por nó, mas tornam o acesso a cada filho $O(1)$ por indexação direta. Alfabetos menores reduziriam o uso de memória por nó, mas poderiam exigir estruturas de acesso mais complexas (como listas encadeadas), aumentando o custo das operações.